

SWYNEX

SECURITY ASSESSMENT REPORT

Application Security Assessment – Task 4

Target: OWASP Juice Shop (localhost assessment environment)

Assessment Period: October 2026

Classification: Security Assessment – Controlled Lab Evidence

1. Executive Summary

This report documents the results of a security assessment performed against an OWASP Juice Shop application running in a controlled local assessment environment. Six security findings were documented from the supplied evidence: SQL Injection, Cross-Site Scripting (XSS), Sensitive Information Exposure, Unauthenticated Administrative Configuration Access, Missing Content Security Policy (CSP), and Web3 Sandbox Exposure.

The findings cover injection handling, output encoding, access control, security configuration, sensitive-data handling, and Web3 functionality. The most security-sensitive issue in the evidence set is the apparent unauthenticated access to an administrative configuration endpoint. The supplied evidence also demonstrates executable XSS and exposed sensitive information in a file/cookie context.

2. Assessment Scope

Item	Details
Application	OWASP Juice Shop
Environment	Controlled local/lab environment
Observed host	localhost / 127.0.0.1
Assessment type	Manual web application security assessment
Evidence	Browser UI, HTTP request/response headers, Network/Debugger views and challenge evidence
Findings documented	6

3. Methodology

- Reviewed application functionality and exposed administrative/Web3 features.
- Inspected browser Network/Debugger evidence and HTTP response headers.
- Validated the supplied XSS evidence by observing JavaScript execution in the application context.
- Reviewed exposed file/resource locations and sensitive browser values shown in the evidence.
- Mapped findings to relevant OWASP Top 10 categories and CWE references.
- Assigned severity based on demonstrated behavior and likely impact; limitations are stated where screenshots alone cannot prove server-side impact.

4. Risk Summary

ID	Finding	Severity	OWASP
F01	SQL Injection	High	A03:2021
F02	Cross-Site Scripting (XSS)	High	A03:2021
F03	Sensitive Information Exposure	High	A02:2021
F04	Unauthenticated Administrative Configuration Access	Critical	A01:2021
F05	Missing Content Security Policy (CSP)	Medium	A05:2021
F06	Web3 Sandbox Exposure	Medium	A05:2021

Severity ratings are assessment judgments based on the supplied evidence and should be validated with server-side testing before being treated as production risk ratings.

F01 — SQL Injection

Severity	High
OWASP Mapping	A03:2021 – Injection
CWE	CWE-89 – Improper Neutralization of Special Elements used in an SQL Command
Evidence	sql-injection.png

Description

The application was tested for SQL injection and the supplied assessment evidence indicates that attacker-controlled input can influence a database query in an unsafe manner.

Impact

Successful SQL injection can allow unauthorized access to application data, authentication bypass in vulnerable query paths, modification or deletion of records, and potentially broader compromise depending on database privileges and application architecture.

Remediation

Use parameterized queries/prepared statements for all database operations; never concatenate untrusted input into SQL. Apply strict server-side input validation, use least-privilege database accounts, suppress verbose database errors, and add automated security tests for injection cases.

Verification

Repeat the original test with representative injection strings and confirm that input is treated as data, database errors are not exposed, and no unauthorized records or query behavior can be triggered.

Evidence Snapshot: sql-injection.png was supplied in the GitHub repository but was not available as a local attachment during document generation; it is therefore referenced by filename.

F02 — Cross-Site Scripting (XSS)

Severity	High
OWASP Mapping	A03:2021 – Injection
CWE	CWE-79 – Improper Neutralization of Input During Web Page Generation
Evidence	xss-attack.png

Description

The evidence shows attacker-controlled HTML/JavaScript entered through the application search functionality and a JavaScript alert executed in the browser. This demonstrates client-side script execution from untrusted input.

Impact

An attacker may execute JavaScript in another user's browser in the affected context. Depending on application protections and the affected page, this can enable phishing, unauthorized actions, DOM manipulation, data access available to the page, or session abuse.

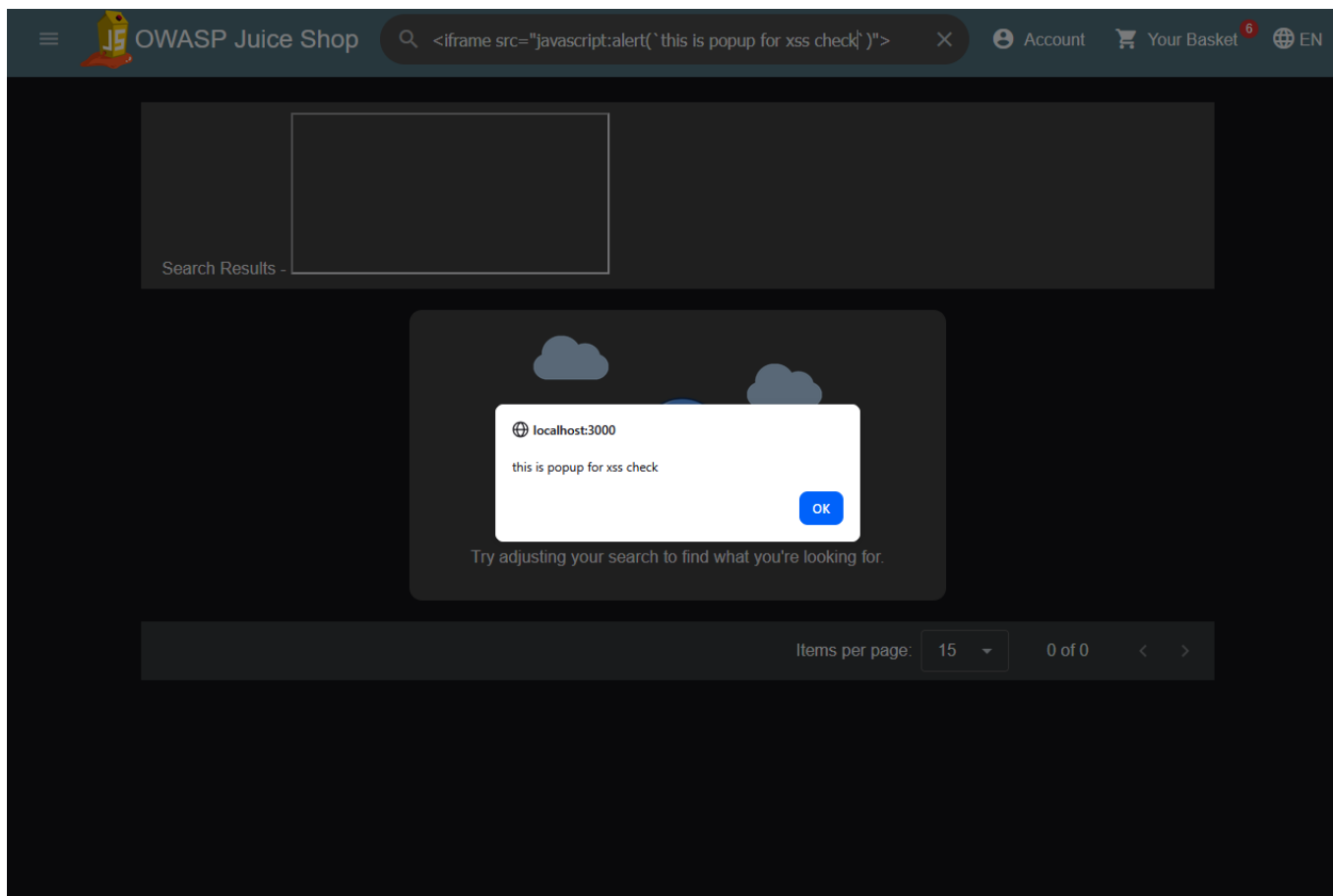
Remediation

Contextually encode untrusted output, use framework-safe templating, sanitize HTML only where HTML is intentionally supported, avoid dangerous DOM sinks such as innerHTML for untrusted data, and deploy a restrictive Content Security Policy.

Verification

Repeat the demonstrated test with benign XSS probes and confirm that the payload is rendered as text or safely sanitized and that no script executes.

Evidence Snapshot



F03 — Sensitive Information Exposure

Severity	High
OWASP Mapping	A02:2021 – Cryptographic Failures / A05:2021 – Security Misconfiguration (context-dependent)
CWE	CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor
Evidence	sensitive-file .png

Description

The evidence shows a browsable FTP directory containing files that appear operational or application-related. The browser's cookie view also displays session-related values in the captured evidence. These values should be treated as sensitive and must not be published in an assessment repository without redaction.

Impact

Public exposure of sensitive files can disclose source, configuration, backup, operational, or other security-relevant information. Exposed session or token material can increase the risk of account/session compromise if the values remain valid.

Remediation

Disable public directory listing where unnecessary, remove sensitive/backup/configuration files from web-accessible locations, enforce authentication and authorization for file repositories, rotate any exposed credentials/tokens, and maintain strict secret-handling procedures for screenshots and reports.

Verification

Confirm that the directory is no longer accessible without authorization, sensitive files cannot be downloaded, and previously exposed tokens/cookies are invalidated.

Evidence Snapshot

quarantine acquisitions.md announcement_encyr...
coupons_2013.md.bak eastere.gg encrypt.pyc
incident-support.kdbx legal.md package-lock.json.bak
package.json.bak suspicious_errors.yml

[illegible]

F04 — Unauthenticated Administrative Configuration Access

Severity	Critical
OWASP Mapping	A01:2021 – Broken Access Control
CWE	CWE-862 – Missing Authorization
Evidence	unauthenticated-admin-config.png

Description

The evidence shows a request to an administrative application-configuration endpoint returning HTTP 200 OK. The endpoint is presented in the assessment as accessible without the required authentication/authorization boundary.

Impact

Unauthorized access to administrative configuration can expose internal settings and may permit modification of security-sensitive application behavior. Depending on the endpoint's write capabilities, this can lead to full application compromise.

Remediation

Enforce server-side authentication and role-based authorization on every administrative endpoint. Do not rely on hidden routes or client-side controls. Apply deny-by-default authorization, validate the authenticated user's role/privileges, and add access-control tests for anonymous and low-privileged users.

Verification

Log out or use an unauthenticated session and confirm the administrative endpoint returns an appropriate authorization response (for example 401/403) and does not disclose configuration data.

Evidence Snapshot

The screenshot displays the OWASP Juice Shop application interface. On the left, a sidebar menu lists various sections: Account, Login, Contact, Customer Feedback, AI Chat (marked as IMPROVED), Company, About Us, Photo Wall, Score Board, and GitHub. The main content area shows a product page for 'Apple Pomace' with a price of 0.89 and an 'Add to Basket' button. On the right, the browser's developer tools are open, showing the Network tab. A GET request to the endpoint `/rest/admin/application-configuration` is highlighted, showing a status of 200 OK. The response headers include `Access-Control-Allow-Origin: *`, `Content-Type: application/json; charset=utf-8`, and other standard headers. The request headers show `Accept: application/json, text/plain, */*` and `Accept-Language: en-US,en;q=0.9`.

F05 — Missing Content Security Policy (CSP)

Severity	Medium
OWASP Mapping	A05:2021 – Security Misconfiguration
CWE	CWE-693 – Protection Mechanism Failure
Evidence	Missing Content Security Policy (CSP) .png

Description

The captured HTTP response headers include common security headers such as X-Content-Type-Options and X-Frame-Options, but no Content-Security-Policy header is visible.

Impact

A missing CSP removes an important browser-side defense-in-depth control. It can increase the impact of certain XSS and content-injection vulnerabilities by allowing a broader set of script or resource execution paths.

Remediation

Deploy a restrictive CSP appropriate to the application, beginning with report-only mode where necessary. Define explicit script-src, object-src, base-uri, frame-ancestors and other required directives; avoid unsafe-inline and unsafe-eval unless there is a documented technical requirement.

Verification

Confirm the response contains a CSP header and test the application to ensure legitimate functionality continues while unauthorized script/resource execution is blocked.

Evidence Snapshot

The screenshot displays the OWASP Juice Shop application interface on the left, featuring a grid of products: Apple Juice (1000ml) for 1.99, Apple Pomace for 0.89, a banana smoothie, and a green smoothie. Each product has an 'Add to Basket' button. On the right, the browser's network inspector is open, showing a GET request to the local host. The 'Response Headers' section is expanded, listing various security headers: Accept-Ranges, Access-Control-Allow-Origin, Cache-Control, Connection, Date, ETag, Feature-Policy, Keep-Alive, Last-Modified, X-Content-Type-Options, X-Frame-Options, and X-Recruiting. Notably, the Content-Security-Policy header is missing from this list.

F06 — Web3 Sandbox Exposure

Severity	Medium
OWASP Mapping	A05:2021 – Security Misconfiguration (context-dependent)
CWE	CWE-284 – Improper Access Control (potentially applicable depending on server-side authorization)
Evidence	broken-access-control.png

Description

The evidence shows a Web3 Code Sandbox exposed in the application with functionality to compile contracts and deploy a selected contract, together with a MetaMask connection interface.

Impact

If this functionality is available to unauthorized users or is insufficiently isolated, it may permit resource abuse, unauthorized blockchain interactions, unexpected transaction costs, or interaction with application-controlled Web3 infrastructure. The screenshot alone does not establish server-side authorization bypass, so impact should be validated against backend controls and wallet/transaction boundaries.

Remediation

Require appropriate authentication and authorization for privileged Web3 operations, isolate sandbox execution, enforce resource and transaction limits, validate contract inputs, use dedicated test wallets with no production secrets, and ensure production keys or privileged RPC credentials are never exposed to the client.

Verification

Test the Web3 functions from an anonymous/low-privileged session and confirm that privileged compilation, deployment, transaction signing, or backend RPC actions are blocked unless explicitly authorized.

Evidence Snapshot

Connect your MetaMask

Web3 Code Sandbox

- Easily compile/deploy and invoke smart contracts from below
- You can pass ETH to the contract both while invoking/deploying by entering the GWEI Value post compilation

Select compiler version

0.8.21

Compile Contract

Contract to deploy

Compiled Contracts

GWEI value for send

0

Deploy selected Contract

Inspector Console Debugger Network 1

Filter URLs

All HTML CSS JS XHR Fonts Images Media WS Other

St...	M...	D...	File	Ini...	Ty...	Tr...	Size
384	GET	...	application-configuration	...	json	ca...	23...
384	GET	...	application-configuration	...	json	ca...	23...
384	GET	...	application-configuration	...	json	ca...	23...
384	GET	...	application-configuration	...	json	ca...	23...
384	GET	...	1	...	json	ca...	15...
200	GET	...	/socket.io/?EIO=4&transport=polling&t=Q3xfBNH&sid=...	...	pl...	23...	1 B
200	P...	...	/socket.io/?EIO=4&transport=polling&t=Q3xfBKS&sid=...	...	ht...	21...	2 B
200	GET	...	/socket.io/?EIO=4&transport=polling&t=Q3xfBKs&sid=...	...	pl...	26...	32...
200	GET	...	/socket.io/?EIO=4&transport=polling&t=Q3xfBFC	...	pl...	32...	96...
384	GET	...	/api/Hints/	...	json	ca...	97...
384	GET	...	/api/Challenges/?sort=name	...	json	ca...	67...
384	GET	...	/api/Challenges/?name=Score Board	...	json	ca...	47...
384	GET	...	/api/Challenges/?name=Score Board	...	json	ca...	47...

24 requests 395.24 kB / 1.50 kB transferred Finish: 2.25 min DOMContentLoaded:

Headers Cookies Request Response Timings Stack Trace

Filter Headers

GET

Scheme: http

Host: 127.0.0.1:3000

Filename: /socket.io/

EIO: 4

transport: polling

t: Q3xfBNH

sid: 4x3IG3fLuUr4-kfoAAAS

Address: 127.0.0.1:3000

Status 200 OK

Version HTTP/1.1

Transferred 230 B (1 B size)

Referrer Policy strict-origin-when-cross-origin

DNS Resolution System

Response Headers (229 B) Raw

Access-Control-Allow-Origin: http://localhost:4200

5. Overall Remediation Plan

- Immediately enforce authentication and authorization on all administrative and configuration endpoints.
- Replace dynamic SQL construction with parameterized queries and add injection regression tests.
- Fix XSS using contextual output encoding/safe templating and by eliminating unsafe DOM sinks.
- Remove sensitive files from publicly reachable locations and disable unnecessary directory listing.
- Rotate/revoke any session tokens, cookies, credentials, or secrets exposed in assessment evidence.
- Deploy and validate a restrictive Content Security Policy.
- Review Web3 sandbox authorization, isolation, transaction controls, wallet boundaries, and production-secret exposure.
- Add automated security tests for access control, injection, XSS, sensitive-file exposure, and security headers.

6. Conclusion

The assessment identified six security findings spanning application input handling, access control, sensitive information management, browser security controls, and Web3 functionality. Remediation should prioritize authorization boundaries and injection/XSS controls, followed by sensitive-data cleanup and security-hardening measures. After fixes are implemented, a targeted re-test should be performed to confirm that the demonstrated behaviors are no longer reproducible.

7. Evidence Repository

The evidence filenames supplied for the assessment repository are listed below. Screenshots containing cookies, tokens, credentials, wallet phrases, or other secrets should be redacted before public publication.

- admin-section.png
- broken-access-control.png
- sensitive-file .png
- sql-injection.png
- unauthenticated-admin-config.png
- xss-attack.png